

Section 10.1: Modern Observability & Telemetry

*Study Guide | 3 Files in This Series | Section 10.**

Executive Overview

Observability is the foundation of production reliability. The three pillars — **logs**, **traces**, and **metrics** — work in concert to give engineers full visibility into system behaviour. This section covers how to instrument, aggregate, and query each pillar, and how they are correlated into a unified debugging workflow.

1. Structured Logging

Why Structure Matters

Unstructured logs are human-readable but machine-hostile. Structured logs treat every log line as a data record, enabling fast indexing, querying, and alerting.

Unstructured (avoid):

```
log.Printf("User 123 placed order for $50")
# Cannot filter by user_id without regex; no schema enforcement
```

Structured (preferred):

```
{
  "timestamp": "2024-06-05T14:30:00Z",
  "level": "INFO",
  "service": "order-service",
  "user_id": "123",
  "order_amount": 50.00,
  "correlation_id": "abc-def-ghi",
  "message": "Order placed successfully"
}
```

Every field is a first-class citizen: indexed, filterable, and aggregatable.

Log Aggregation Pipeline

A typical pipeline collects logs from multiple sources, normalises them, and ships them to a searchable store.

Fluent Bit (collector/forwarder):

```
[INPUT]
  Name      tail
  Path      /var/log/app/*.log
  Parser    json
  Tag        app.*

[OUTPUT]
  Name      kafka
  Match      app.*
  Brokers    kafka:9092
  Topics     application-logs
```

Elasticsearch index schema:

```
Index: application-logs-YYYY.MM.DD
Fields:
  user_id      → keyword  (exact match, faceting)
  order_amount → float    (range queries)
  correlation_id → keyword (exact match – critical for tracing)
  message      → text     (full-text search)
  timestamp    → date     (time-range queries)
```

Pipeline comparison:

Component	Role	When to use
Fluent Bit	Lightweight collector	Edge/container environments, low memory
Logstash	Full ETL pipeline	Complex transformations, enrichment
Vector	High-throughput router	Polyglot environments, schema conversion
Fluentd	Plugin-rich aggregator	Large plugin ecosystem needed

2. Distributed Tracing

Distributed tracing follows a single request as it crosses service boundaries, recording timing and causality at each hop.

Trace Propagation

W3C TraceContext standard headers:

```
TraceParent: 00-0af7651916cd43dd8448eb211c803da8-b7ad6b7169203331-01
      ↑   ↑                               ↑           ↑
      ver Trace ID (128-bit)         Parent Span ID   Flags
```

The `traceparent` header is forwarded by every participating service, creating a chain that spans the full request lifetime.

Span Model

Spans are the atomic unit of a trace. Parent-child relationships express causality.

```
Trace A:
Span 1: HTTP GET /orders      [web-service]    0ms → 85ms
Span 2: Database query        [order-db]     5ms → 40ms
Span 3: Cache lookup          [redis]       5ms → 12ms
Span 4: Payment call          [payment-svc] 45ms → 80ms
Span 5: Email send            [email-svc]   82ms → 85ms (async)
```

The waterfall view immediately shows that the database query dominates latency.

Sampling Strategies — When to Use Each

Strategy	Mechanism	Pros	Cons	Best for
Head-based	Decision at trace start (random %)	Simple, low overhead	Misses rare errors	High-volume, mostly-healthy services
Tail-based	Buffer all spans; decide after completion	Captures errors & slow traces	Memory/CPU overhead	Latency-sensitive or error-prone services
Adaptive	Dynamically adjust rate by endpoint	Balances cost vs coverage	Complex config	Mixed-criticality endpoints

Rule of thumb: Use head-based sampling at 1–5% for baseline coverage; layer tail-based sampling to ensure 100% of error traces are kept.

3. Metrics & Monitoring

Prometheus Architecture

Prometheus uses a **pull model**: the server scrapes `/metrics` endpoints at a configurable interval (typically 15–30 seconds). This inverts responsibility — services expose metrics; Prometheus collects them.

```
# Exposition format
# HELP http_requests_total Total HTTP requests received
# TYPE http_requests_total counter
http_requests_total{method="GET",status="200"} 1234
http_requests_total{method="POST",status="500"} 7
```

Service discovery (Kubernetes example):

```
scrape_configs:
  - job_name: 'kubernetes-pods'
    kubernetes_sd_configs:
      - role: pod
    relabel_configs:
      - source_labels: [__meta_kubernetes_pod_annotation_prometheus_io_scrape]
        action: keep
        regex: true
```

PromQL — Key Query Patterns

```
# 99th-percentile latency over the last hour
histogram_quantile(
  0.99,
  rate(http_request_duration_seconds_bucket[1h])
)

# Alert: p99 latency > 100ms for 5 consecutive minutes
histogram_quantile(
  0.99,
  rate(http_request_duration_seconds_bucket[5m])
) > 0.1

# Error rate as a percentage of total traffic
sum(rate(http_requests_total{status=~"5.."}[5m]))
```

```
/
sum(rate(http_requests_total[5m])) * 100
```

The Four Golden Signals

These four metrics, when monitored together, surface the vast majority of production issues:

Signal	What it measures	Example metric
Latency	Time to serve a request (success vs error separately)	<code>http_request_duration_seconds</code>
Traffic	Demand on the system	<code>http_requests_total (req/s)</code>
Errors	Rate of failed requests	<code>http_requests_total{status=~"5.."} </code>
Saturation	How “full” a resource is	<code>container_cpu_usage_seconds_total</code>

Key insight: Monitoring only availability (uptime) misses latency degradation, traffic spikes, and partial failures. All four signals are needed.

4. Observability Pillars — Correlation & Comparison

When to Use Each Pillar

Pillar	Granularity	Retention cost	Best for
Metrics	Aggregated	Low (timeseries, no payload)	Dashboards, SLO tracking, alerting
Logs	Per-event	Medium	Debugging specific errors, audit trail
Traces	Per-request	High (many spans)	Latency root-cause, dependency mapping

A complete investigation workflow: 1. **Alert fires** on a metric (error rate > 1%) 2. **Dashboard** shows which service degraded 3. **Logs** reveal the error message and affected `correlation_id`s 4. **Trace** shows exactly which downstream call caused the failure

5. Interview Problems

Problem 1 — Design an Observability Stack for 50 Microservices

Scenario: You are the lead SRE for an e-commerce platform with 50 services. A customer reports their order was not processed. You have no idea which service failed.

Solution:

Step 1 — Inject correlation IDs at the edge:

```
class RequestIDMiddleware:
    def __call__(self, environ, start_response):
        correlation_id = (
            environ.get('HTTP_X_CORRELATION_ID') or
            generate_uuid()
        )
        g.correlation_id = correlation_id

        def wrapped_start_response(status, headers):
            headers.append(('X-Correlation-ID', correlation_id))
            return start_response(status, headers)

        return self.app(environ, wrapped_start_response)
```

Step 2 — Structured logging with mandatory context:

```
import structlog

logger = structlog.get_logger()

def log_with_context(level, message, **kwargs):
    logger.log(level, message,
               correlation_id=g.correlation_id,
               service="order-service",
               **kwargs
    )

log_with_context("info", "Order created", order_id=123)
log_with_context("info", "Payment processed", amount=50.00)
```

Step 3 — OpenTelemetry auto-instrumentation:

```
from opentelemetry.instrumentation.flask import FlaskInstrumentor
```

```
FlaskInstrumentor().instrument_app(app)
# All outbound HTTP calls automatically propagate traceparent header
```

Step 4 — Query across all 50 services:

```
Elasticsearch / Kibana:
correlation_id: "abc-def-ghi"
AND service: *           ← matches all 50 services
Sort by: timestamp asc   ← builds a timeline
```

Step 5 — Retention strategy (cost-optimised):

```
Logs:    30 days → S3 Glacier after 90 days
Traces:   7 days → expensive; keep only sampled + error traces
Metrics:  2 years → cheap timeseries; keep everything
```

Key insight: Correlation IDs are the glue binding all three pillars. The cost of propagating them (a few bytes per request) is negligible compared to the debugging time saved.

Problem 2 — Alert on p99 Latency Without False Positives

Scenario: Your current alert fires every deploy because the rolling window briefly spikes. Engineers are ignoring it (“alert fatigue”). Fix it.

Root cause: `rate()[5m]` over-reacts to short spikes.

Solution — Multi-window burn rate alerting:

```
# Fast burn: severe issue — page now
(
  histogram_quantile(0.99, rate(http_request_duration_seconds_bucket[5m])) > 0.5
)
AND
(
  histogram_quantile(0.99, rate(http_request_duration_seconds_bucket[1h])) > 0.2
)

# Slow burn: degradation trend — ticket (not page)
histogram_quantile(0.99, rate(http_request_duration_seconds_bucket[6h])) > 0.15
```

Why dual-window works: A 5-minute spike alone is a deploy blip. A 5-minute spike *and* elevated 1-hour average means real degradation. Both must be true to page.

Problem 3 — Sampling Strategy for a Checkout Service

Scenario: Your checkout service handles 50,000 requests/minute. Full tracing would cost \$12,000/month. You need to keep errors visible while reducing cost by 95%.

Solution — Tiered sampling:

```
# OpenTelemetry Collector config
processors:
  tail_sampling:
    decision_wait: 10s
    policies:
      - name: errors-policy
        type: status_code
        status_code: {status_codes: [ERROR]}
        # Result: 100% of error traces kept

      - name: slow-policy
        type: latency
        latency: {threshold_ms: 500}
        # Result: 100% of slow traces kept

      - name: probabilistic-baseline
        type: probabilistic
        probabilistic: {sampling_percentage: 2}
        # Result: 2% of healthy traces kept
```

Cost outcome: 100% errors + 100% slow + 2% normal $\approx$ 5% overall → 95% cost reduction with zero loss of actionable signal.

Quick-Reference Cheat Sheet

```
Structured log → JSON with correlation_id + service + timestamp
Trace propagation → W3C traceparent header forwarded by every service
Sampling → Head-based (cheap) + tail-based errors (comprehensive)
Four golden signals → Latency · Traffic · Errors · Saturation
PromQL p99 → histogram_quantile(0.99, rate(bucket[window]))
Retention → Metrics (2yr) > Logs (30d) > Traces (7d)
```